Laboratório Prolog · Aula 13

Matheus Goulart de Alencar · RA 241319952 · Turma ESOFT6SB
Gerado em 20/10/2026, 20:05:33 · tempo registrado no cronômetro: 23:42

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

A primeira consulta procura quais valores estão associados a funcional, enquanto a segunda verifica se java está associado a logico.

Saída do ola.pl

```
?- linguagem(_).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)

?- paradigma(prolog, _).
   paradigma(prolog, logico)

?- paradigma(_, funcional).
   paradigma(scheme, funcional)

?- paradigma(java, logico).
   false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P): pergunta qual é o paradigma da linguagem Prolog.
Resposta: P = logico

paradigma(prolog, logico): verifica se Prolog pertence ao paradigma lógico.
Resposta: true

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Significa que, na base de fatos do programa, java não está associado ao paradigma lógico, o fato cadastrado é paradigma(java, orientado_a_objetos) portanto a consulta retorna false

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem): 7 respostas — lucia, marta, tereza, joana, bia, ana e dora.

(mae(M, F), mulher(F)): 4 pares — (lucia, tereza), (marta, joana), (tereza, bia) e (joana, ana).

Saída do familia.pl

```
?- mae(joana, F1).
   mae(joana, jake)
   mae(joana, ana)

?- pai(P2, jake).
```

```
    pai(jorge, jake)

?- (mae(M3, F3), homem(F3)).
   mae(lucia, vern)
   mae(marta, paulo)
   mae(joana, jake)
   mae(dora, leo)

?- (homem(X4), mulher(X4)).
   false.

?- (mae(joana, jake), mae(joana, ana)).
   (mae(joana, jake), mae(joana, ana))
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Significa: “Quais são as mães e suas filhas que são mulheres?” Teve 7 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

O pressuposto do mundo fechado: o Prolog considera falso aquilo que não pode provar com os fatos e regras da base.

Isso pode ser um problema em situações reais porque não ter informação não significa que algo seja falso. Maria pode ser pai de Jake, mas, como isso não está registrado na base, o Prolog retorna false.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

A ordem foi: lucia, marta, tereza, joana, bia, ana, dora.

A ordem é determinada pela sequência em que os fatos mulher/1 aparecem na base de conhecimento, pois o Prolog os procura de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake): false (nenhum tio satisfaz as regras e os fatos).

primo(leo, X): false (nenhum primo satisfaz as regras e os fatos).

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Vão falhar:

f(X, X) = f(a, b) — falha, pois X teria que ser a e b ao mesmo tempo.

[A|B] = [] — falha, pois uma lista vazia não tem cabeça nem cauda.

2 + 3 = 5 — falha, pois Prolog não calcula a expressão automaticamente durante a unificação.

f(X, X) = f(a, Y) não falha: unifica com X = a e Y = a.

Saídas de regras.pl e unificacao.pl

```
?- irmao(X1, Y1).
   irmao(joana, paulo)
   irmao(paulo, joana)

?- tio(X2, Y2).
   false.

?- ancestral(X3, leo).
   ancestral(dora, leo)
   ancestral(paulo, leo)
   ancestral(lucia, leo)
   ancestral(osvaldo, leo)
   ancestral(marta, leo)
   ancestral(vern, leo)

?- primo(X4, Y4).
   false.

[vern, tereza, joana, paulo, bia, jake, ana, leo]

---

--- constantes e variaveis
a = a    unifica
a = b    NAO unifica
_ = a    unifica
--- estruturas
f(_,_)=f(a,_)    unifica
f(_,_)=f(a,b)    NAO unifica
f(_,_)=f(a,_)    unifica
pai(_,jake)=pai(jorge,_)    unifica
--- listas: [Cabeca|Cauda]
[_|_]=[1,2,3]    unifica
[_|_]=[]    NAO unifica
[_,_|_]=[a,b,c,d]    unifica
--- aritmetica NAO e unificacao
2+3 = 5    NAO unifica
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

Duas cláusulas: para definir o caso direto e o indireto de ancestralidade.

Caso base: ancestral(X, Y) :- progenitor(X, Y). — X é progenitor direto de Y.

Caso recursivo: ancestral(X, Y) :- progenitor(X, Z), ancestral(Z, Y). — X é ancestral de Y se for progenitor de Z e Z for ancestral de Y.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a regra passa a considerar uma pessoa irmã de si mesma. A meta é necessária para garantir que X e Y sejam diferentes, evitando esse resultado incorreto.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

f(X, X) = f(a, b): não unifica porque X teria que ser a e b ao mesmo tempo.

f(X, X) = f(a, Y): unifica com X = a e Y = a, pois os dois argumentos ficam iguais.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Prolog, a regra avo permite consultar tanto os avós quanto os netos, mudando as variáveis da consulta. Em Java ou Scheme, normalmente seria necessário criar funções específicas para cada operação. Prolog usa relações lógicas, que podem ser consultadas em diferentes direções, enquanto funções tradicionais recebem entradas e calculam resultados.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X, Y, [1, 2, 3]) terá 4 respostas:

X = [], Y = [1, 2, 3]
X = [1], Y = [2, 3]
X = [1, 2], Y = [3]
X = [1, 2, 3], Y = []

Saída do listas.pl

```
?- concatena(X1, [b, c], [a, b, c]).
   concatena([a], [b, c], [a, b, c])

?- inverte([a, b, c, d], R2).
   inverte([a, b, c, d], [d, c, b, a])

?- tamanho([x, y], N3).
   tamanho([x, y], 2)

?- membro(X4, []).
   false.
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([], 0) — lista vazia tem tamanho 0.

Passo recursivo: tamanho([_|Cauda], N) :- tamanho(Cauda, M), N is M + 1. — calcula o tamanho da cauda e soma 1.

Para tamanho([a,b,c], N), ficam pendentes os cálculos de N, M e das próximas caudas até chegar a []. Depois, o Prolog retorna: N = 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Há 4 respostas porque a lista [1, 2, 3] pode ser dividida em duas listas de 4 maneiras diferentes.

O append do Scheme normalmente recebe duas listas e calcula a lista resultante. Já concatena do Prolog pode funcionar em várias direções: descobrir a primeira lista, a segunda ou testar se a concatenação é válida.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

O backtracking (retrocesso): o Prolog procura todas as soluções possíveis, voltando e testando a próxima alternativa até encontrar todas as respostas.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2 + 3: X recebe a estrutura 2+3, sem calcular.

X is 2 + 3: X vale 5.

fatorial(N, 120): dá erro de instanciação, pois N está livre.

Saída do aritmetica.pl

```
?- X1 is 5 * 4 + 2.
   X1 = 22

?- X2 = 5 * 4 + 2.
   X2 = 5*4+2

?- fatorial(10, F3).
   F3 = 3628800

?- fib(20, F4).
   F4 = 6765

?- fatorial(N5, 120).
   ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica, sem calcular: X = 2 + 3 → X = 2+3.

is calcula: X is 2 + 3 → X = 5.

=:= compara valores numéricos: 5 =:= 2 + 3 → true.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

concatena funciona em várias direções porque usa unificação. Já fatorial(N, 120) dá erro porque a regra usa operações aritméticas (> e is) que exigem que N tenha um valor conhecido. A aritmética do Prolog não é totalmente relacional como a concatenação de listas.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: ambos usam recursão, com caso base (0! = 1) e caso recursivo (N! = N × (N−1)!).

Diferente: em Haskell, fact n retorna o resultado; em Prolog, fatorial(N, F) relaciona a entrada N ao resultado F, usando is para calcular.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1(X) e solteiro2(X): preciso ver as regras para saber qual dará false.

maximo2(5, 3, M): duas respostas, M = 5 e M = 3, pois a segunda cláusula aceita qualquer valor.

maximo3(5, 3, 3): false, pois o máximo correto é 5.

Saída de controle.pl e mensagem de ancestral_ruim

```
--- A) backtracking: acompanhe a ordem das tentativas
?- mesmo_gosto(X, Y).
   tenta ana e ana
   tenta ana e bia
   tenta ana e caio
   resposta: mesmo_gosto(ana, caio)
   tenta bia e ana
   tenta bia e bia
   tenta bia e caio
   tenta caio e ana
   resposta: mesmo_gosto(caio, ana)
   tenta caio e bia
   tenta caio e caio
--- B) negacao: a ordem das metas muda a resposta
?- solteiro1(_).
   solteiro1(osvaldo)
   solteiro1(paulo)
   solteiro1(jake)
?- solteiro2(_).
   false.
--- C) corte
?- maximo1(5, 3, _).
   maximo1(5, 3, 5)
?- maximo2(5, 3, _).
   maximo2(5, 3, 5)
   maximo2(5, 3, 3)
?- maximo3(5, 3, _).
   maximo3(5, 3, 5)
?- maximo1(5, 3, 3).
   false.
?- maximo3(5, 3, 3).
   false.

---

ancestral_bom(osvaldo, jake)?
  sim
ancestral_ruim(osvaldo, jake)? (vai estourar a pilha)
Fatal Error: local stack overflow
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores: 24 colorações.

Com 2 cores: 0 colorações, pois PR, SP e MS fazem fronteira entre si e precisam de 3 cores diferentes.

Saída do mapa.pl

```
--- coloracoes com 3 cores, na ordem [RS, SC, PR, SP, MS, MG, RJ, ES]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,verde,azul,vermelho,azul,vermelho]
--- quantas coloracoes existem com 3 cores?
24
```

```
--- e com 2 cores?
?- mapa([vermelho,verde],_).
   false.
--- e com 4 cores?
768
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

O Prolog encontrou mesmo_gosto(ana, caio) e mesmo_gosto(caio, ana).

Em tenta ana e ana, a meta X \== Y falhou, pois X e Y eram iguais. O Prolog voltou por backtracking para a escolha de amigo(Y) e tentou o próximo amigo, Bia, depois Caio.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro2(X) dá false porque \+ casado(X) é avaliado antes de X receber um valor. Já solteiro1(X) primeiro busca os homens e depois verifica se não estão casados.

Osvaldo é considerado solteiro porque não existe o fato casado(osvaldo) na base. Ser pai de alguém não significa, para o Prolog, ser casado.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte ! impede testar outras cláusulas, mas não verifica se o terceiro argumento é o máximo correto. Como a cabeça unifica com M = 3 só depois, pode aceitar um resultado errado.

Correção: teste os valores e faça a unificação do resultado depois do corte:

maximo3(X, Y, M) :- X >= Y, !, M = X.
maximo3(X, Y, M) :- M = Y.
Assim, maximo3(5, 3, 3) retorna false.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim estoura a pilha porque chama a si mesma antes de verificar progenitor, entrando em recursão infinita.

ancestral_bom verifica primeiro progenitor(X, Z) e depois faz a chamada recursiva, avançando pela árvore familiar até encontrar o caso base. No Prolog, a ordem das metas importa, mesmo que as regras expressem a mesma relação lógica.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

3 cores: 24 colorações.

2 cores: 0 colorações.

4 cores: 768 colorações.

É declarativo porque descreve as condições para uma coloração válida, sem especificar um algoritmo ou laço. O Prolog usa backtracking para encontrar as soluções automaticamente.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma lógico, concatena define uma relação que permite juntar ou descobrir listas em várias direções.
No funcional, anexar recebe duas listas e calcula uma nova lista como resultado.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Ordem das regras e metas: ancestral_ruim entrou em recursão infinita e estourou a pilha porque a chamada recursiva vinha primeiro.

Aritmética não relacional: fatorial(N, 120) gerou erro porque N estava livre e o Prolog precisava de seu valor para calcular.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de lógica, como sistemas especialistas, relações familiares e coloração de mapas, pois permite definir fatos e regras e buscar soluções automaticamente.

Não escolheria Prolog para aplicações com muitas operações numéricas, interfaces gráficas ou controle detalhado do programa; nesses casos, Java costuma ser mais adequado.

Exercícios opcionais

Opcionais: resultado dos testes

```
% Exercicio 1
irma(X, Y) :- irmao(X, Y), mulher(X).
tia(X, Y) :- irmao(X, P), mulher(X), progenitor(P, Y).

% Exercicio 2
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).

% Exercicio 3
soma_quadrados([], 0).
soma_quadrados([H|T], S) :-
    soma_quadrados(T, S1),
    S is S1 + H * H.

pares([], []).
pares([H|T], [H|P]) :-
    H mod 2 =:= 0,
    pares(T, P).
pares([H|T], P) :-
    H mod 2 =\= 0,
    pares(T, P).

% Exercicio 4
mdc(A, 0, A) :- A > 0.
mdc(A, B, D) :-
    B > 0,
    R is A mod B,
    mdc(B, R, D).

potencia(_, 0, 1).
```

```
potencia(B, E, R) :-
    E > 0,
    E1 is E - 1,
    potencia(B, E1, R1),
    R is B * R1.

% Exercicio 5
caminho(A, B) :- caminho_aux(A, B, [A]).
caminho_aux(A, B, _) :- liga(A, B).
caminho_aux(A, B, Visitados) :-
    liga(A, C),
    \+ membro(C, Visitados),
    caminho_aux(C, B, [C|Visitados]).

% Exercicio 6
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta),
    membro(SC, Paleta),
    membro(PR, Paleta),
    RS \== SC,
    SC \== PR.

% Exercicio 7
colega(X, Y) :-
    funcionario(X, D, _),
    funcionario(Y, D, _),
    X \== Y.

ganha_mais(X, Y) :-
    funcionario(X, _, S1),
    funcionario(Y, _, S2),
    S1 > S2.

total_depto(D, Total) :-
    findall(S, funcionario(_, D, S), Salarios),
    soma(Salarios, Total).
```

E1 Regras de família: código escrito

```
irma(X, Y) :-
    mulher(X),
    irmao(X, Y).

tia(X, Y) :-
    mulher(X),
    progenitor(P, Y),
    irmao(X, P).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|Cauda], X) :-
    ultimo(Cauda, X).
```

E3 As funções da Aula 12: código escrito

```
% Soma dos quadrados dos elementos da lista
soma_quadrados([], 0).
soma_quadrados([H|T], S) :-
    soma_quadrados(T, S1),
    S is S1 + H * H.

% Retorna uma lista contendo apenas os elementos pares
pares([], []).
pares([H|T], [H|R]) :-
    H mod 2 =:= 0,
    pares(T, R).
pares([H|T], R) :-
    H mod 2 =\= 0,
    pares(T, R).
```

E4 Aritmética: código escrito

```
% Maximo divisor comum usando o algoritmo de Euclides
mdc(A, 0, A).
mdc(A, B, D) :-
    B > 0,
    R is A mod B,
    mdc(B, R, D).

% Potencia usando recursao
potencia(_, 0, 1).
potencia(Base, Expoente, R) :-
    Expoente > 0,
    E1 is Expoente - 1,
    potencia(Base, E1, R1),
    R is Base * R1.
```

E5 Grafo com ciclos: código escrito

```
% Verifica se existe um caminho entre duas cidades,
% evitando visitar a mesma cidade mais de uma vez.

caminho(A, B) :-
    caminho_visitados(A, B, [A]).

caminho_visitados(A, B, _) :-
    liga(A, B).

caminho_visitados(A, B, Visitados) :-
    liga(A, C),
    \+ membro(C, Visitados),
    caminho_visitados(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
% Coloracao do mapa do Sul: RS, SC e PR.
% Regioes vizinhas devem ter cores diferentes.

mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta),
    membro(SC, Paleta),
    membro(PR, Paleta),
```

```
    RS \== SC,
    SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
% X e Y trabalham no mesmo departamento, mas sao pessoas diferentes.
colega(X, Y) :-
    funcionario(X, D, _),
    funcionario(Y, D, _),
    X \== Y.

% X ganha mais que Y.
ganha_mais(X, Y) :-
    funcionario(X, _, SalarioX),
    funcionario(Y, _, SalarioY),
    SalarioX > SalarioY.

% Soma os salarios de todos os funcionarios do departamento D.
total_depto(D, Total) :-
    findall(Salario, funcionario(_, D, Salario), Lista),
    soma(Lista, Total)
```